

Tracing and debugging

How each tool is switched on, what a run looks like, and what happens when the agent breaks.

Turning tracing on

This is the biggest difference, and it is visible in the code.

LangSmith: one environment variable

```
LANGSMITH_TRACING=true
```

That is it. No imports, no code changes, nothing passed into the agent.

Proof is in the run list. The rows named `LangGraph` came from running the plain app:

```
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"
```

That file does not mention LangSmith anywhere. It still shows up with full latency, token and cost data.

Personal / Tracing / ilm-observability-lab

ilm-observability-lab ID Tracing Evaluators Automations Insights Engine New

Default View Last 1 day Threads Traces Runs Input Search input content

Add filter <>

2

nts 1

Beta

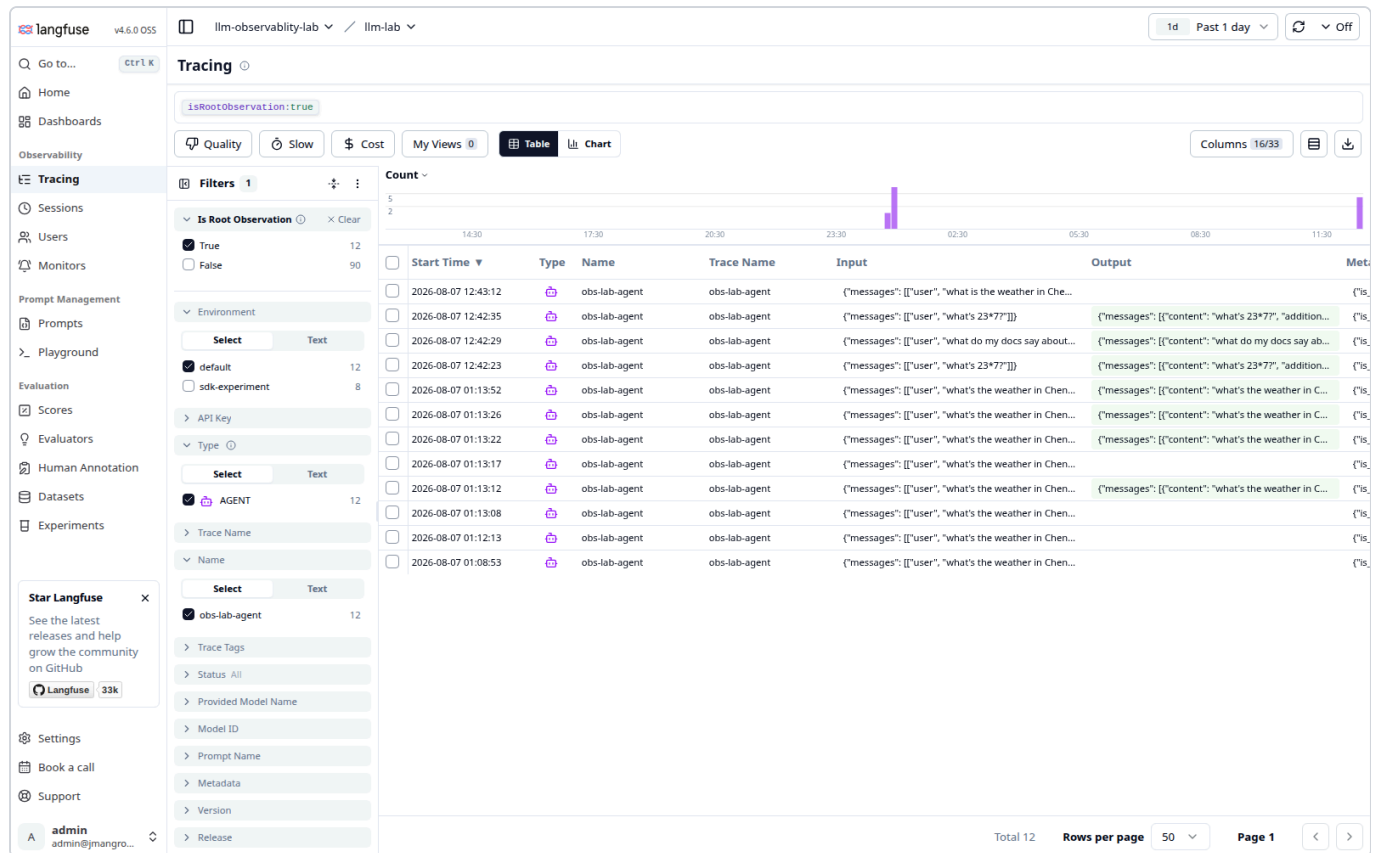
☐	☒	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags
☐	forced-weather-failure	user: what is the weather ...			RuntimeError("weather s...	8/7/2026, 11:51...	1.36s			279	\$0.0008175		
☐	LangGraph	user: what's the weather ...	ai: The weather in Chenn...			8/7/2026, 11:4...	2.13s			690	\$0.002295		
☐	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...			8/7/2026, 11:41...	2.77s			690	\$0.002295		phase3
☐	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...			8/7/2026, 11:35...	3.96s			878	\$0.002795		
☐	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...			8/7/2026, 11:3...	2.18s			690	\$0.002295		manual-te
☐	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...			8/7/2026, 11:17...	0.54s			130	\$0.0004675		
☐	LangGraph	user: what's 23*7?	ai: 23 * 7 equals 161.			8/7/2026, 11:17...	0.99s			577	\$0.001645		
☐	LangGraph	user: what's the weather ...			RuntimeError("weather s...	8/7/2026, 11:17...	0.63s			317	\$0.0011525		
☐	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...			8/7/2026, 11:17...	0.62s			236	\$0.0008225		
☐	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...			8/7/2026, 11:17...	3.26s			878	\$0.002795		
☐	LangGraph	user: weather in Mumbai,...			RuntimeError("weather s...	8/7/2026, 11:17...	1.41s			282	\$0.0008175		
☐	LangGraph	user: what's the weather ...			RuntimeError("weather s...	8/7/2026, 11:15...	0.66s			317	\$0.0011525		
☐	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...			8/7/2026, 11:15...	0.56s			133	\$0.000475		
☐	LangGraph	user: what's 23*7?	ai: The result of 23 * 7 is 1...			8/7/2026, 11:15...	1.11s			581	\$0.001685		
☐	ChatOpenAI	human: You are grading ...	ai: First line: CORRECT Se...			8/7/2026, 11:15...	0.78s			288	\$0.0009975		
☐	LangGraph	user: what do my docs s...	ai: Your docs say: - Retry ...			8/7/2026, 11:15...	14.07s			924	\$0.003255		

Look at the columns you get for free: `Latency` , `Tokens` , `Cost` , `Error` , `Tags` . The red rows are the weather tool failing. Nobody wrote error handling to make those appear.

Langfuse: one handler per call

```
handler = CallbackHandler()
graph.invoke(..., config={"callbacks": [handler]})
```

Nothing is recorded unless you pass that handler.



Which is better depends on what you care about. LangSmith gets you tracing in ten seconds, but it also captures everything in the process, including code you did not write. Langfuse costs you a line at every call site, and in exchange nothing leaves the process unless you asked for it.

One useful side effect: with both sets of keys in `.env`, a single run lands in **both** tools at once. Worth doing once.

What a single run looks like

Both give you the same core thing: a tree of what happened, with time and tokens on each node.

LangSmith

The screenshot shows the LangGraph interface. On the left, a table lists traces with columns for Name, Input, and status. The main panel displays a detailed trace for 'LangGraph' at 11:44 AM. The trace shows a sequence of steps: 'agent' (1.55s), 'ChatOpenAI' (1.26s), 'should_continue' (0.00s), 'tools' (0.00s), 'get_weather' (0.00s), 'calculator' (0.00s), and another 'agent' (0.57s). The right panel shows the 'Attributes' section, including 'Metadata' with fields like 'revision_id', 'LANGSMITH_ENDPOINT', 'LANGSMITH_PROJECT', 'LANGSMITH_TRACING', 'Is_integration', 'Is_run_depth', and 'Runtime' with fields like 'langchain_core_version', 'langchain_version', 'library', 'library_version', 'platform', 'py_implementation', 'runtime', 'runtime_version', 'sdk', and 'sdk_version'.

2.13s , 690 tokens, \$0.0023 at the root. The graph:step and seq:step labels come from LangGraph and show the execution order.

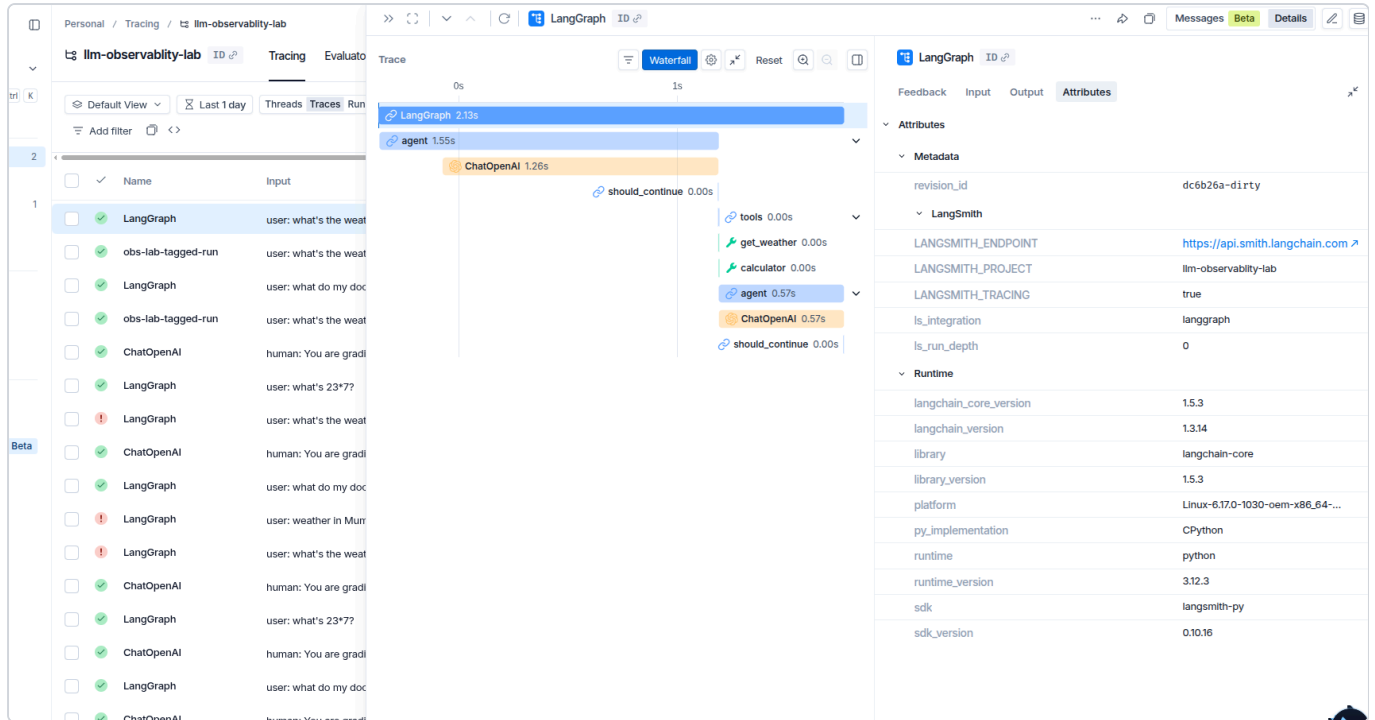
Langfuse

The screenshot shows the Langfuse interface. The left sidebar contains navigation options like Home, Dashboards, Observability, Tracing, Sessions, Users, Monitors, Prompt Management, Prompts, Playground, Evaluation, Scores, Evaluators, Human Annotation, Datasets, and Experiments. The main panel displays a trace for 'obs-lab-agent: 696c2cfd5732276e3ecb0cb08aecedab3'. The trace tree shows a sequence of steps: 'obs-lab-agent' (1.87s), 'agent' (1.35s), 'ChatOpenAI' (1.15s), 'should_continue' (0.00s), 'tools' (0.00s), 'calculator' (0.00s), 'agent' (0.51s), 'ChatOpenAI' (0.51s), and 'should_continue' (0.00s). The right panel shows the 'Preview' section with 'Input' (Path, Value), 'Calculator' (161), 'Assistant' (23 * 7 equals 161), 'Corrected Output', and 'Metadata' (Path, Value).

Same shape, plus a rendered graph of the agent loop underneath the tree.

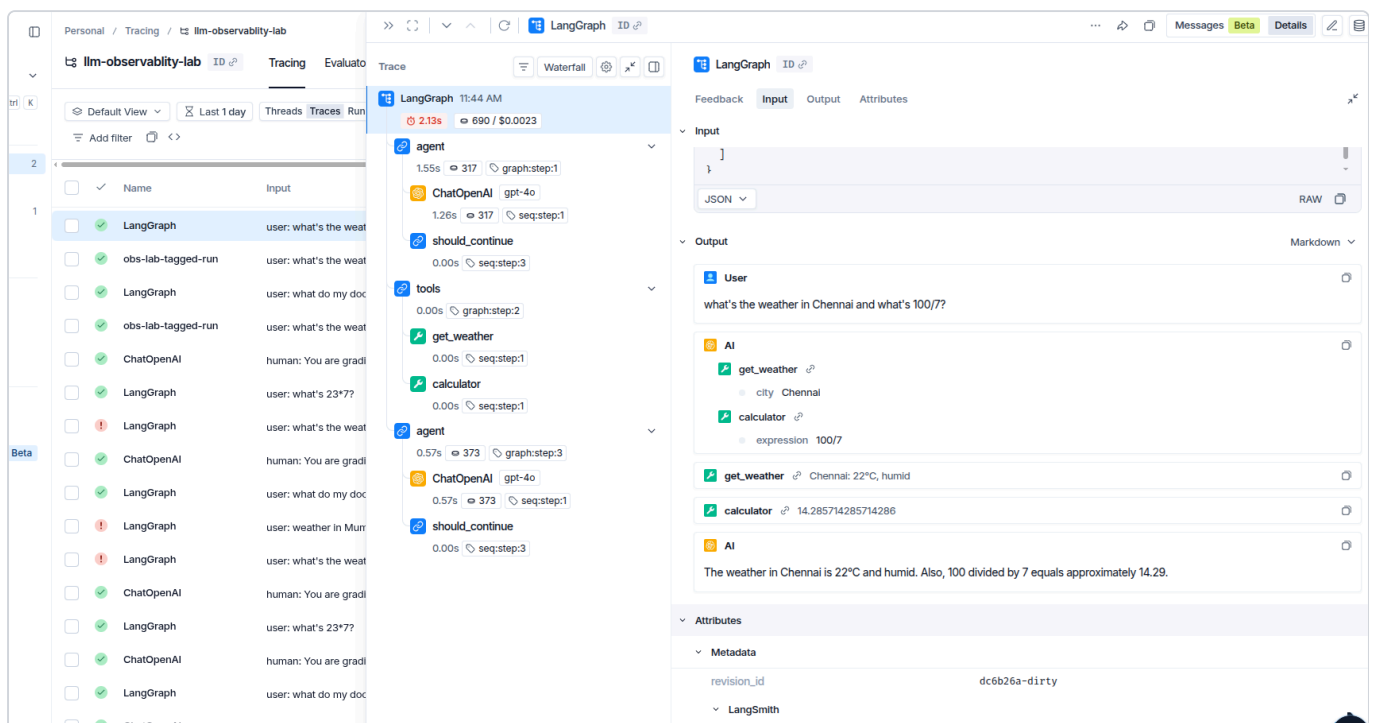
Timing view

LangSmith has a waterfall that makes it obvious where the time actually went:



The whole run is 2.13s and the `ChatOpenAI` call is 1.26s of it. Both tool calls are 0.00s. The model is the slow part, not your code. That is nearly always the answer, and it is worth seeing once.

The conversation itself



Both tools show the full message flow: what the model asked for, what each tool returned, what came back at the end.

When something breaks

This is where the two tools genuinely differ, and it matters more than any feature list.

LangSmith marks the root of the trace and puts the full Python traceback in a dedicated **Error** tab. Output says **No outputs**.

The screenshot displays the LangSmith interface for a trace titled "forced-weather-failure". The left sidebar shows a list of traces, with "forced-weather-failure" selected. The main panel shows the trace details, including a tree view of the execution steps and a detailed view of the error.

Trace Tree:

- forced-weather-failure (11:51 AM, 1.36s, 279 / \$0.0009)
 - agent (1.34s, 279, graph:step:1)
 - ChatOpenAI (1.06s, 279, seq:step:1)
 - should_continue (0.00s, seq:step:3)
 - tools (0.01s, graph:step:2)
 - get_weather (0.00s, seq:step:1)

Error Tab:

```
Error RuntimeError("weather service timeout for city='Chennai'")Traceback (most recent call last):
  File "/home/vikash-kumar/Deve...
```

Feedback Tab:

+ Add feedback

Input Tab:

```
{
  "messages": [
    {
      "user",
      "what is the weather in Chennai?"
    }
  ]
}
```

Output Tab:

No outputs

Attributes Tab:

- Metadata
 - revision_id: dc6b26a-dirty
 - LangSmith
 - LANGSMITH_ENDPOINT: <https://api.smith.langchain.com>

Langfuse marks every span the error passed through, all the way up.

Read the Langfuse tree carefully:

```
obs-lab-agent  ERROR
  agent
    ChatOpenAI      1.30s   270 -> 48   $0.001155
    should_continue
    tools            ERROR
      get_weather    ERROR    0.00s
      calculator     0.00s
```

`get_weather` failed, so `tools` is marked, so the whole trace is marked. `calculator` sits right next to it, untouched and green. The graph below even paints `__end__` red.

Verdict on errors: Langfuse wins. LangSmith tells you *that* the run failed and hands you a traceback. Langfuse shows you *where* in the agent it failed without you reading a single line of Python.

One detail worth knowing: `get_weather` shows `0.00s`. A real network timeout would take seconds. Zero means it failed on the first line of the function. That is how you tell a fake failure from a real one.

Grouping runs together

Both can thread separate runs into one conversation.

LangSmith calls them Threads:

The screenshot shows the LangSmith interface for a thread named 'llm-observability-lab'. The interface is divided into several sections:

- Left Sidebar:** Contains navigation options like 'Experiments', 'Queues', 'Tracing', and 'Evaluation'. It also shows a list of runs with columns for 'Name' and 'Input'.
- Top Bar:** Displays the current thread name 'llm-observability-lab' and various controls like 'Default View', 'Last 1 day', and 'Threads'.
- Main View:** Shows a detailed view of a specific run. It includes a 'Turns' section with a list of turns (1-4) and a 'Thread ID' field. The 'Inputs' section shows a user prompt: 'what's the weather in Chennai and what's 100/7?'. The 'Outputs' section shows the AI's response, including tool calls like 'get_weather' and 'calculator'.
- Right Panel:** Contains a 'Stats' section with metrics like 'First start', 'Last end', 'Latency', and 'Total cost breakdown'.

Langfuse calls them Sessions:

The screenshot shows the Langfuse interface for a session named 'llm-observability-lab'. The interface is divided into several sections:

- Left Sidebar:** Contains navigation options like 'Home', 'Dashboards', 'Observability', 'Tracing', 'Sessions', 'Users', 'Monitors', 'Prompt Management', 'Evaluation', 'Scores', 'Evaluators', 'Human Annotation', 'Datasets', and 'Experiments'.
- Top Bar:** Displays the current session name 'llm-observability-lab' and various controls like '1d', 'Past 1 day', and 'Columns'.
- Main View:** Shows a table of sessions. The table has columns for 'ID', 'Created At', 'Duration', 'Environment', 'Scores', 'User IDs', and 'Traces'. A single session is listed with ID 'session-001', created at '2026-08-07 01:08:53', duration '11h 34m 20s', environment 'default', user 'vikash', and 12 traces.
- Filter Panel:** Located on the left, it allows filtering by 'Environment' (default, sdk-experiment), 'Session ID', 'User IDs', 'Trace Tags', 'Metadata', 'Bookmarked' (Bookmarked, Not bookmarked), 'Session Duration', 'Traces Count', 'Input Tokens', 'Output Tokens', 'Total Tokens', 'Input Cost', 'Output Cost', 'Total Cost', 'Categorical Scores', 'Numeric Scores', 'Boolean Scores', 'Comment Count', and 'Comment Content'.

One session, session-001 , holding 12 traces over 11 hours, attributed to user vikash .

That 11 hour duration is a warning, not a feature. The session ID is hardcoded in [traced_run.py](#), so every run ever executed piles into the same session. In a real app you generate one per conversation.

The two tools name these fields differently:

	LangSmith	Langfuse
Tags	<code>config["tags"]</code>	<code>metadata["langfuse_tags"]</code>
User	<code>metadata["user_id"]</code> , any key you like	<code>metadata["langfuse_user_id"]</code> , reserved name
Session	<code>metadata["session_id"]</code> , any key you like	<code>metadata["langfuse_session_id"]</code> , reserved name

LangSmith lets you invent your own keys and filter on them. Langfuse only promotes specific `langfuse_` - prefixed keys into real UI features, and treats everything else as plain metadata. You trade magic strings for purpose-built screens.
